



Processos

Prof. Pedro Botelho

Universidade Federal do Ceará

26 de março de 2026

Conteúdo



Introdução a Processos

Evolução dos Processos

Criação e Finalização

Características dos Processos

Modelagem de Multiprogramação

Exercícios



Introdução a Processos

Evolução dos Processos

Criação e Finalização

Características dos Processos

Modelagem de Multiprogramação

Exercícios



Programa

- ▶ Código (sequência de instruções) para tratar um problema
- ▶ São aplicações ou utilitários
- ▶ Conceito **estático**, sem estado interno

Exemplos de programas:

- ▶ C:\Windows\notepad.exe
- ▶ /usr/bin/firefox



Processo (ou Tarefa)

- ▶ Execução das instruções definidas no programa
- ▶ Conceito **dinâmico**, com estado interno
- ▶ Estado interno evolui a cada instante
- ▶ Implementada de várias formas (processos, threads, ...)

Exemplos de processos:

- ▶ O Notepad editando um arquivo readme.txt
- ▶ O visualizador de PDFs mostrando este slide



- ▶ Computadores modernos → várias ações simultâneas
- ▶ Exemplo: Servidor Web
 - ▶ Solicitações de páginas
 - ▶ Cache ou leitura do disco
 - ▶ Acesso ao disco muito lenta (visão da CPU)
 - ▶ Solicitações durante leitura do disco
- ▶ Necessidade de modelar e controlar
- ▶ Chaveamento da CPU: ilusão de paralelismo

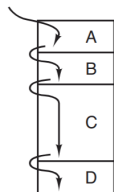


- ▶ Processo → atividade (informações relacionadas):
 - ▶ Programa (lista de instruções)
 - ▶ Contador de programa (endereço de instrução)
 - ▶ Registradores
 - ▶ Variáveis
- ▶ Apenas um contador de programa real (físico)
- ▶ Contador lógico carregado no físico
- ▶ Escalonamento: regras de controle
- ▶ Dois programas iguais → processos distintos

Múltiplos Processos



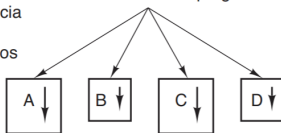
Um contador de programa



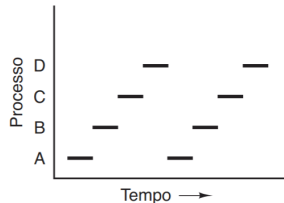
(a)

Alternância
entre
processos

Quatro contadores de programa



(b)



(c)

- (a) Multiprogramação de quatro programas.
- (b) Modelo conceitual de quatro processos sequenciais independentes.
- (c) Apenas um programa está ativo de cada vez.

Conteúdo



Introdução a Processos

Evolução dos Processos

Criação e Finalização

Características dos Processos

Modelagem de Multiprogramação

Exercícios

Sistemas Monotarefa

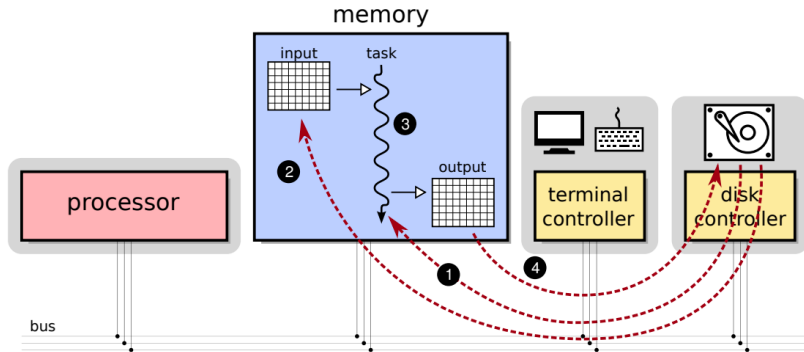


Usado nos primeiros sistemas de computação.

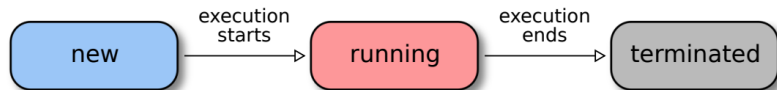
Executam uma única tarefa de cada vez:

- (1) O programa é carregado do disco para a memória
- (2) Os dados do programa são carregado na memória
- (3) O programa executa até sua conclusão
- (4) Os resultados do programa são salvos
- (5) Repete para o próximo programa

Diagrama do Sistema Monotarefa



Estados de uma Tarefa



Por que Sistemas Multitarefa?



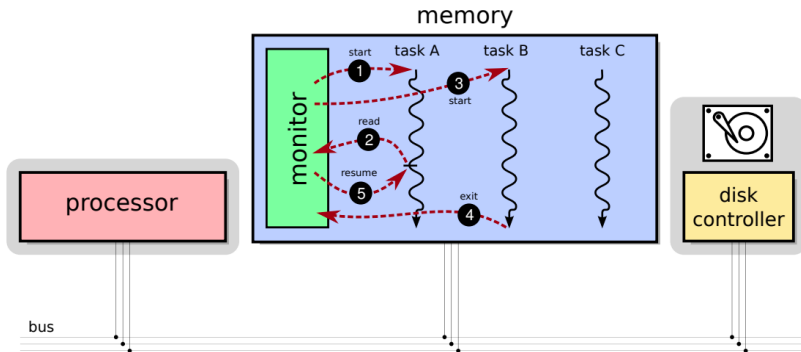
Problemas nos sistemas monotarefa:

- ▶ Tarefas esperando por entrada/saída ficam paradas
- ▶ Custo de operação do computador era muito elevado
- ▶ Ex: UNIVAC I (1951) consumia 125 kW, o equivalente a 500 PCs

Solução:

- ▶ Carregar várias tarefas na memória
- ▶ Usar o processador ocioso para tratar outras tarefas
- ▶ Um *software* **monitor** coordena a troca de tarefas

Diagrama do Sistema Multitarefa



Problemas em Sistemas Multitarefa



Problemas dos sistemas multitarefa simples:

- ▶ Aplicações em laço infinito podem bloquear o sistema
- ▶ Aplicações interativas não funcionam bem

```
1  int main () {  
2      int i = 0, soma = 0 ;  
3  
4      while (i < 1000)  
5          soma += i ; // erro: i nao foi incrementado  
6  
7      printf ("A soma vale %d\n", soma);  
8  }
```



Solução: *Time Sharing* ou **preempção** por tempo

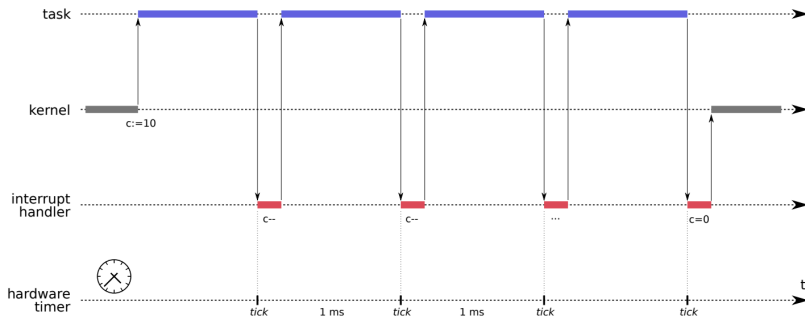
- ▶ Cada tarefa recebe uma fatia de tempo (*quantum*) de CPU
- ▶ A tarefa perde a CPU ao acabar seu quantum
- ▶ *Quantum* típico vai de 10 ms a 200 ms
- ▶ Implementado através de interrupções (ticks)

Funcionamento dos Sistemas Preemptivos



- (1) A tarefa recebe o processador
- (2) A tarefa executa até expirar seu quantum ou encerrar
- (3) A tarefa é interrompida pelo hardware
- (4) A tarefa retorna ao estado “pronto”
- (5) A Outra tarefa recebe o processador

Implementação do Tempo Compartilhado:



Conteúdo



Introdução a Processos

Evolução dos Processos

Criação e Finalização

Características dos Processos

Modelagem de Multiprogramação

Exercícios



- ▶ Sistemas simples → ao ser ligado
- ▶ Sistemas de propósito geral: 4 eventos possíveis

(1) Início do sistema

- ▶ Foreground x Background (daemons)

(2) Por outro processo

- ▶ Divisão de tarefas

(3) Pelo usuário

- ▶ Comando ou clique no mouse

(4) Tarefa em lote (*batch*)

- ▶ Submissão de tarefas



- ▶ Em todos os casos ocorre uma chamada de sistema
 - ▶ Pai inicia processo Filho (**fork**)
 - ▶ Processo Filho executa programa (**execve**)
- ▶ Porque dois passos?
 - ▶ Permite filho executar ações (entre fork e execve)
 - ▶ Redirecionar entradas e saídas (padrão e de erros)
- ▶ Windows: apenas um passo (CreateProcess)
- ▶ Espaços de endereçamento distintos
 - ▶ Filho uma cópia do pai inicialmente
 - ▶ Não compartilhada



- ▶ 4 eventos possíveis
- ▶ Saída normal (voluntário)
 - ▶ Trabalho concluído
 - ▶ Chamada informa ao SO (exit no UNIX e ExitProcess no Windows)
 - ▶ Exemplo: compilação terminada ou comando do usuário
- ▶ Saída por erro (voluntário)
 - ▶ Programa encontra um erro
 - ▶ Raro quando baseados em janela
 - ▶ Exemplo: compilar arquivo inexistente



- ▶ Erro fatal (involuntário)
 - ▶ Erros no programa
 - ▶ Instrução ilegal
 - ▶ Tratamento de exceção: sinalização ao SO
 - ▶ Exemplo: referência a memória inexistente, divisão por zero
- ▶ Por outro processo (involuntário)
 - ▶ Chamada para finalizar processo
 - ▶ Exemplo: kill (UNIX) e TerminateProcess (Windows)
- ▶ Alguns casos: filhos finalizados junto com o pai

Conteúdo



Introdução a Processos

Evolução dos Processos

Criação e Finalização

Características dos Processos

Modelagem de Multiprogramação

Exercícios

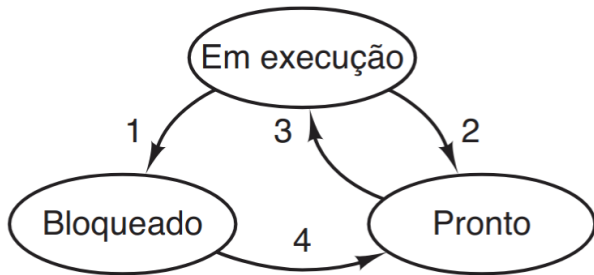
Hierarquia de Processos



- ▶ Associação entre pais e filhos
- ▶ UNIX: grupos de processos
- ▶ Exemplo:
 - ▶ Sinal do teclado enviado a um grupo de processos
 - ▶ Individualmente cada processo trata (ou ignora) o sinal
- ▶ UNIX:
 - ▶ Processo *init* na carga do sistema
 - ▶ Cada usuário em um terminal pode iniciar novos processos
 - ▶ Todos processos dentro da mesma árvore: *init* na raiz



- ▶ Pode depender de interações interprocessos
 - ▶ Exemplo: `cat chapter1 chapter2 chapter3 | grep tree`
- ▶ Processo não executando:
 - ▶ Está pronto mas não tem CPU
 - ▶ Tem CPU mas não está pronto
- ▶ Processo executando: tem CPU e o que processar
- ▶ Modelagem básica: 3 estados e 4 transições
 - ▶ Em execução (running): usando CPU
 - ▶ Pronto (ready): pronto para usar a CPU
 - ▶ Bloqueado (blocked): a espera de um evento

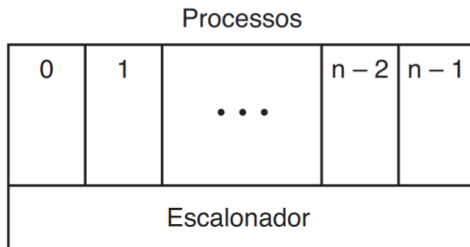


1. O processo é bloqueado aguardando uma entrada
2. O escalonador seleciona outro processo
3. O escalonador seleciona esse processo
4. A entrada torna-se disponível

Estados de Processos



- ▶ Modelo de gerenciamento: Escalonador
- ▶ Ocultamento de detalhes





- ▶ SO mantém uma tabela chamada de **tabela de processos**
- ▶ Estrutura de dados
- ▶ Tabela de processos (**Process Control Block**)
- ▶ Informações importantes do estado do processo:
 - ▶ Contador de programa
 - ▶ Ponteiro da pilha
 - ▶ Alocação de memória
 - ▶ Estados dos arquivos abertos
- ▶ Salvo a informação quando o processo é trocado de estado

Tabela de processos



Gerenciamento de processo	Gerenciamento de memória	Gerenciamento de arquivo
Registros Contador de programa Palavra de estado do programa Ponteiro da pilha Estado do processo Prioridade Parâmetros de escalonamento ID do processo Processo pai Grupo de processo Sinais Momento em que um processo foi iniciado Tempo de CPU usado Tempo de CPU do processo filho Tempo do alarme seguinte	Ponteiro para informações sobre o segmento de texto Ponteiro para informações sobre o segmento de dados Ponteiro para informações sobre o segmento de pilha	Diretório-raiz Diretório de trabalho Descritores de arquivo ID do usuário ID do grupo

Sequência da Troca de Processos



1. O hardware empilha o contador de programa etc.
2. O hardware carrega o novo contador de programa a partir do arranjo de interrupções.
3. O vetor de interrupções em linguagem de montagem salva os registradores.
4. O procedimento em linguagem de montagem configura uma nova pilha.
5. O serviço de interrupção em C executa (em geral lê e armazena temporariamente a entrada).
6. O escalonador decide qual processo é o próximo a executar.
7. O procedimento em C retorna para o código em linguagem de montagem.
8. O procedimento em linguagem de montagem inicia o novo processo atual.

Conteúdo



Introdução a Processos

Evolução dos Processos

Criação e Finalização

Características dos Processos

Modelagem de Multiprogramação

Exercícios

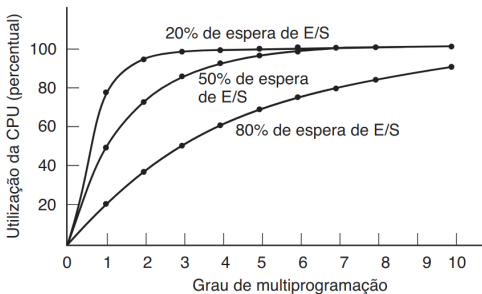


- ▶ Objetivo: aumento da utilização da CPU
- ▶ Processo médio passa 20% do tempo computando
- ▶ 5 processos $\rightarrow$ 100% de ocupação da CPU
 - ▶ Muito simplista
 - ▶ Otimista
 - ▶ Pouco realista
 - ▶ Ignora espera de E/S

Tempo Esperando E/S



- ▶ Modelagem probabilística
- ▶ Fração do tempo esperando E/S = p
- ▶ Probabilidade de CPU ociosa com n processos = p^n
- ▶ Utilização da CPU = $1 - p^n$





- ▶ Modelo aproximado
- ▶ Assume processos independentes
- ▶ Uso em previsões
- ▶ Exemplo:
 - ▶ Memória 512MB
 - ▶ Ocupação: SO 128MB e cada programa 128MB
 - ▶ 3 programas na memória
 - ▶ Fração de espera por E/S: 80%
 - ▶ Ocupação de 49%
 - ▶ 512MB + 512MB $\rightarrow$ 7 programas , ocupação 79% (+30%)
 - ▶ 1024MB + 512MB $\rightarrow$ 12 programas , ocupação 91% (+12%)

Conteúdo



Introdução a Processos

Evolução dos Processos

Criação e Finalização

Características dos Processos

Modelagem de Multiprogramação

Exercícios



- 1 Defina o conceito de processo.
- 2 Por que o conceito de processo é tão importante no projeto de sistemas multiprocessáveis?
- 3 É possível que um programa execute no contexto de um processo e não execute no contexto de um outro? Por que?
- 4 Como o sistema operacional implementa o conceito de processo? Qual a estrutura de dados indicada para organizar os diversos processos na memória principal?
- 5 Defina os cinco estados possíveis de um processo?
- 6 Diferencie processos multithreads, subprocessos e processos independentes.



1 Defina o conceito de processo.

Resposta: Um processo pode ser definido como o ambiente onde um programa é executado. Este ambiente, além das informações sobre a execução, possui também o quanto de recursos do sistema cada programa pode utilizar, como o espaço de endereçamento, tempo de processador e área em disco.

2 Por que o conceito de processo é tão importante no projeto de sistemas multiprocessáveis?

Resposta: Através de processos, um programa pode alocar recursos, compartilhar dados, trocar informações e sincronizar sua execução. Nos sistemas multiprogramáveis os processos são executados concorrentemente, compartilhando o uso do processador, memória principal, dispositivos de E/S dentre outros recursos.

3 É possível que um programa execute no contexto de um processo e não execute no contexto de um outro? Por que?

Resposta: Sim, pois a execução de um programa pode necessitar de recursos do sistema que um processo pode possuir enquanto outro não.



- 4 Como o sistema operacional implementa o conceito de processo? Qual a estrutura de dados indicada para organizar os diversos processos na memória principal?

Resposta: O processo é implementado pelo sistema operacional através de uma estrutura de dados chamada bloco de controle do processo (Process Control Block — PCB). A partir do PCB, o sistema operacional mantém todas as informações sobre o contexto de hardware, contexto de software e espaço de endereçamento de cada processo.



5 Defina os três estados possíveis de um processo?

Resposta: **Estado de Execução:** processo que está sendo processado pela UCP no momento. **Estado de Pronto:** processo que aguarda para ser executado. **Estado de Espera:** processo que aguarda por algum evento ou recurso para prosseguir processamento.



6 Diferencie processos multithreads, subprocessos e processos independentes.

Resposta:

Processos independentes não têm vínculo com os processos criadores. A criação de um processo independente exige a alocação de um PCB, possuindo contextos de hardware, contexto de software e espaço de endereçamento próprios.

Subprocessos são processos criados dentro de uma estrutura hierárquica. Caso um processo pai deixe de existir, os subprocessos subordinados são automaticamente eliminados. Semelhante aos processos independentes, subprocessos possuem seu próprio PCB. Além da dependência hierárquica entre processos e subprocessos, uma outra característica neste tipo de implementação é que subprocessos podem compartilhar quotas com o processo pai. Neste caso, quando um subprocesso é criado, o processo pai cede parte de suas quotas ao processo filho.



- 6 **Processos multithreads** suportam múltiplos threads, cada qual associado a uma parte do código da aplicação. Neste caso não é necessário haver diversos processos para a implementação da concorrência. Threads compartilham o processador da mesma maneira que um processo, ou seja, enquanto um thread espera por uma operação de E/S, outro thread pode ser executado .